

Internet stvari i servisi

Aleksa Stanković 19370

1. Opis sistema

Sistem se sastoji od tri nezavisna API servisa — REST, gRPC i GraphQL — implementiranih iznad iste PostgreSQL baze podataka koja sadrži senzorske podatke IoT uređaja. Svaki servis izlaže iste operacije: dohvaćanje poslednjih N očitavanja po uređaju i agregaciju po satima.

- Baza: PostgreSQL sa ~10.000 zapisa po uređaju
- REST: ASP.NET Core Web API
- gRPC: ASP.NET Core + Protobuf
- GraphQL: Apollo Server (Node.js)
- Load testing: k6 (Grafana)
- Monitoring: Grafana + Prometheus

2. Scenariji testiranja

Definisana su tri scenarija koji pokrivaju različite obrasce pristupa podacima karakteristične za IoT sisteme:

Scenario A — Upis podataka (Write)

Simulira kontinuirano slanje merenja sa senzora. Svaka iteracija šalje novo očitavanje sa svim poljima (temperatura, vlažnost, pritisak, svetlost, zvuk, pokret, baterija) za nasumično odabrani uređaj — putem REST POST zahteva, GraphQL mutacije, odnosno gRPC IngestReading poziva.

Scenario B — Selektivno čitanje (Selective Read)

Dohvata poslednjih 10 očitavanja za nasumični uređaj. GraphQL vraća isključivo polja timestamp, temperature i humidity, čime demonstrira eliminaciju over-fetchinga. REST endpoint /selective/{deviceId} vraća ograničen skup polja. gRPC, međutim, vraća kompletan SensorReading message sa svim poljima — delimična selekcija polja nije podržana na nivou Protobuf protokola, što predstavlja inherentno ograničenje i relevantan nalaz komparativne analize.

Scenario C — Agregacija (Heavy Aggregation)

Zahteva agregatne vrednosti po satima za 2024. godinu: prosečne vrednosti temperature, vlažnosti, pritiska, svetlosti i zvuka, ukupan broj motion evenata, minimalnu vrednost baterije i ukupan broj očitavanja. REST i GraphQL eksplicitno šalju vremenski opseg, dok gRPC k6 test ne prosleđuje from/to parametre — server primenjuje podrazumevane vrednosti. Scenario simulira analitički upit visokog računskog opterećenja.

3. Implementacija sistema

Sistem je realizovan kao skup od tri nezavisna mikroservisa iznad zajedničke PostgreSQL baze. Svi servisi su kontejnerizovani i orkestrirani putem Docker Compose-a, čime je obezbeđen identičan runtime okvir za sva tri protokola i ponovljivost testova.

3.1 Šema baze podataka

Baza je dizajnirana kao time-series struktura sa dve tabele: devices (registar uređaja) i sensor_readings (vremenski niz očitavanja). Svako očitavanje sadrži pet senzorskih vrednosti (temperatura, vlažnost, pritisak, svetlost, zvuk), dodatno motion event i nivo baterije, kao i timestamp i referencu na uređaj.

Da bi se obezbedile efikasne upite tipičnih za IoT radna opterećenja, definisana su četiri indeksa:

1. `idx_readings_device_time (device_id, timestamp DESC)` — primarno za dohvaćanje poslednjih N očitavanja po uređaju (Scenario B).
2. `idx_readings_timestamp (timestamp)` — za vremenske range upite i agregacije po satima (Scenario C).
3. `idx_readings_device (device_id)` — za grupisanje i filtriranje po uređaju.
4. `idx_readings_location (location)` — za filtriranje po lokaciji uređaja.

Pored toga, definisan je view `hourly_aggregates` koji enkapsulira agregaciju po satima i uređajima (prosečne vrednosti senzorskih polja, broj motion evenata, minimum baterije, ukupan broj očitavanja), čime se logika Scenarija C izdvaja iz aplikativnog sloja u bazu.

3.2 REST servis

REST servis je implementiran u ASP.NET Core Web API-ju i izlaže četiri endpointa pod `/api/readings` — GET (lista), POST (upis), GET `/selective/{deviceId}` (selektivni read sa ograničenim skupom polja) i GET `/aggregates` (agregacija po satima). Servis automatski generiše OpenAPI 3.0 specifikaciju (Swagger UI), što omogućava interaktivnu dokumentaciju i klijentsko generisanje.

3.3 gRPC servis

RPC servis je takođe implementiran u ASP.NET Core-u, sa strogo definisanim ugovorom preko `.proto` fajla. Servis `SensorService` izlaže tri RPC metode: `IngestReading` za pojedinačan upis, `GetReadings` za dohvaćanje skupa očitavanja i `GetAggregates` za agregatne upite. Sve poruke (`SensorReading`, `AggregateBucket`) definisane su kao Protobuf strukture, što obezbeđuje determinističku serijalizaciju i mali payload, ali eliminiše mogućnost selekcije polja na nivou protokola.

3.4 GraphQL servis

GraphQL servis je implementiran u Apollo Server-u (Node.js). Šema definiše tipove `SensorReading` i `HourlyAggregate`, kao i Query (`readings`, `aggregates`, `devices`) i Mutation (`ingestReading`). Klijent eksplicitno bira polja u upitu, što direktno eliminiše over-fetching i čini GraphQL pogodnim za Scenario B.

3.5 Docker Compose okruženje

Cela infrastruktura je definisana u jednom `docker-compose.yml` fajlu. Pokreće se sedam servisa: PostgreSQL baza, tri aplikativna kontejnera (`rest-service:5001`, `grpc-service:5002`, `graphql-service:5003`), `cAdvisor:8080` za prikupljanje metrika po kontejneru, `Prometheus:9090` za skladištenje vremenskih nizova i `Grafana:3000` za vizualizaciju. Komunikacija među servisima ide preko interne Docker mreže, dok su portovi eksponirani prema hostu radi pokretanja k6 testova.

4. Metodologija testiranja

Testiranje je sprovedeno korišćenjem k6 alata (Grafana) sa tri nivoa opterećenja kako bi se analiziralo ponašanje sistema pod različitim brojem istovremenih korisnika:

Nivo	Virtualni korisnici (VUs)	Trajanje
Nizak	10	20s
Srednji	100	20s
Visok	500	20s

Svaki scenario testiran je nezavisno za svaku od tri paradigme (REST, gRPC, GraphQL), što daje ukupno 27 test konfiguracija (3 scenarija × 3 paradigme × 3 nivoa opterećenja).

Pragovi (thresholds):

- http_req_duration — latencija zahteva (avg, p90, p95)
- http_req_failed — stopa grešaka
- iterations/s — propusnost (throughput)
- data_received / data_sent — mrežni saobraćaj

Kriterijumi prihvata:

- p(95) latencije < 2000ms
- Stopa grešaka < 1%
- Uspešnost > 99%

Pokretanje testova

docker-compose up -d

Svi testovi pokrenuti su sekvencijalno korišćenjem shell skripte koja automatski iterira kroz sve kombinacije scenarija, protokola i nivoa opterećenja:

```
for SCENARIO in a b c; do
  for PROTOCOL in rest grpc graphql; do
    for VUS in 10 100 500; do
      k6 run \
        --out experimental-prometheus-rw \
        -e K6_PROMETHEUS_RW_SERVER_URL=http://localhost:9090/api/v1/write \
        -e VUS=$VUS \
        -e DURATION=20s \
        --tag scenario=$SCENARIO \
        --tag protocol=$PROTOCOL \
        --summary-export results/${SCENARIO}-${PROTOCOL}-${VUS}.json \
        scenario-${SCENARIO}-${PROTOCOL}.js
      sleep 5
    done
  done
done
```

Između svakog testa uvedena je pauza od 10 sekundi kako bi se server vratio u stabilan state pre sledećeg testa.

5. Rezultati testiranja

Svi testovi izvedeni su pod istim uslovima (20s trajanja, isti hardver, ista mreža) za sve tri paradigme i tri nivoa opterećenja (10, 100, 500 VUs). Tabele u nastavku prikazuju ključne metrike po scenariju: prosečnu i p95 latenciju, propusnost (RPS), broj iteracija, primljene i poslate podatke, kao i stopu grešaka. Latencija za REST i GraphQL preuzeta je iz metrike http_req_duration, dok je za gRPC iz grpc_req_duration.

Scenario A — Pojedinačno upisivanje (Single Insert)

Protokol	VUs	Avg (ms)	p95 (ms)	RPS	Iteracije	Recv (MB)	Sent (MB)	Greške
REST	10	1.31	1.92	7461	149,235	33.6	52.9	0.0%
REST	100	5.93	10.49	16732	334,720	75.3	118.6	0.0%
REST	500	28.29	38.56	17608	352,535	79.3	124.9	0.0%
gRPC	10	1.53	2.22	6314	126,291	7.1	19.0	0.0%
gRPC	100	6.64	11.72	14917	298,421	16.9	44.8	0.0%
gRPC	500	31.82	45.45	15640	313,131	18.0	47.1	0.0%
GraphQL	10	1.32	2.07	7387	147,745	38.3	79.1	0.0%
GraphQL	100	12.78	16.97	7795	155,993	40.5	83.5	0.0%
GraphQL	500	68.54	86.12	7273	145,737	37.8	78.0	0.0%

Scenario B — Selektivno čitanje (Selective Read)

Protokol	VUs	Avg (ms)	p95 (ms)	RPS	Iteracije	Recv (MB)	Sent (MB)	Greške
REST	10	2.70	4.21	3342	66,853	653.3	6.5	0.0%
REST	100	20.17	33.83	4808	96,222	940.2	9.3	0.0%
REST	500	101.32	150.18	4887	97,917	956.8	9.5	0.0%
gRPC	10	1.43	2.25	6510	130,222	122.1	10.5	0.0%
gRPC	100	8.31	14.36	11794	235,986	221.3	19.1	0.0%
gRPC	500	40.19	66.55	12312	246,537	231.5	20.0	0.0%
GraphQL	10	1.57	2.27	6186	123,728	121.5	34.6	0.0%
GraphQL	100	16.23	19.26	6141	122,905	120.7	34.3	0.0%
GraphQL	500	87.20	97.53	5711	114,554	112.5	32.0	0.0%

Scenario C — Agregacija (Heavy Aggregation)

Protokol	VUs	Avg (ms)	p95 (ms)	RPS	Iteracije	Recv (MB)	Sent (MB)	Greške
REST	10	41.28	57.76	237	4,845	0.5	0.7	99.6%
REST	100	269.57	493.58	366	7,462	0.8	1.2	100.0%
REST	500	1278.46	2218.15	374	8,088	0.9	1.2	100.0%
gRPC	10	55.95	75.36	175	3,573	0.4	0.2	99.0%
gRPC	100	333.67	613.13	288	6,042	0.2	0.3	100.0%
gRPC	500	1781.47	3072.99	260	5,788	0.4	0.3	100.0%
GraphQL	10	53.20	69.04	181	3,765	3.6	1.3	49.5%
GraphQL	100	529.93	1875.91	184	3,823	3.6	1.3	49.5%
GraphQL	500	2504.60	9944.66	180	4,254	4.1	1.4	49.5%

Napomena: U Scenariju C svi protokoli su pokazali izuzetno visoke stope grešaka, što ukazuje da serverska strana (PostgreSQL + servisni sloj) nije mogla da održi računski intenzivnu agregaciju po satima nad celom 2024. godinom pod paralelnim opterećenjem od 100 i 500 VU. Stoga apsolutne vrednosti latencije i propusnosti u ovom scenariju treba tumačiti sa rezervom — one više odražavaju ponašanje pod zasićenjem nego stabilno performansno stanje.

6. Komparativna analiza

Tabele iz prethodne sekcije pokazuju jasne razlike između tri paradigme. U nastavku se razmatraju tri aspekta koja su najrelevantnija za IoT kontekst: veličina mrežnog saobraćaja (JSON vs Protobuf), efekat over-fetching-a u operacijama čitanja, i ograničenja vezana za stabilnost servera pod opterećenjem.

6.1 Veličina podataka: JSON vs Protobuf

Najjasnija razlika između protokola vidljiva je u količini prenesenih podataka. Tabela ispod prikazuje prosečnu veličinu odgovora po zahtevu (data_received / iterations) pri opterećenju od 100 VU:

Scenario	REST (JSON)	GraphQL (JSON)	gRPC (Protobuf)	gRPC vs REST
A — Single Insert	236 B	272 B	59 B	−75%
B — Selective Read	10,246 B	1,030 B	983 B	−90%
C — Aggregation (response)	111 B	996 B	37 B	−67%

Protobuf (binarni format koji koristi gRPC) konzistentno proizvodi najmanje payload-e. U scenariju A, gRPC odgovor je 4× manji od REST-a (59 B naspram 236 B), iako oba prenose istu logičku informaciju — potvrdu upisa. U scenariju B razlika je još drastičnija: REST vraća kompletan dokument senzorskih očitavanja kao JSON (~10 KB), dok gRPC isti podskup polja prenosi u manje od 1 KB.

Ova razlika direktno utiče na propusnost — u Scenariju B, gRPC postiže 11,794 RPS naspram 4,808 RPS za REST pri istih 100 VU, jer manji odgovori brže oslobađaju resurse i mreže i klijenta. Za IoT uređaje na granici mreže, gde je propusni opseg često ograničen (LoRaWAN, NB-IoT, mobilne veze) a potrošnja energije proporcionalna količini prenetih bajtova, smanjenje payload-a od 75–90% predstavlja značajnu prednost gRPC/Protobuf rešenja. Trošak ove uštede je gubitak

ljudske čitljivosti — Protobuf poruke nisu čitljive bez .proto šeme, dok se JSON može direktno inspektovati u logovima i alatima.

6.2 Over-fetching u operacijama čitanja

Scenario B je dizajniran tako da klijent traži samo podskup polja iz senzorskog očitavanja. Razlika između paradigmi u ovom slučaju je strukturalna, ne samo optimizaciona. REST endpoint vraća kompletan resurs — ceo dokument očitavanja sa svim poljima — bez obzira na to koja polja klijent zaista koristi. GraphQL i gRPC, sa druge strane, omogućavaju klijentu da specifikuje tačno koja polja želi.

Praktičan efekat ovog over-fetching-a vidi se u kolonama Recv (MB) za Scenario B. Pri 100 VU, REST je primio 940 MB podataka, GraphQL 121 MB, a gRPC 221 MB. REST prenosi približno 7.8× više bajtova od GraphQL-a i 4.3× više od gRPC-a — što direktno smanjuje njegovu propusnost (4,808 RPS vs 6,141 RPS za GraphQL vs 11,794 RPS za gRPC).

Drugim rečima, REST ne pati od inferiornog protokola, već od nesposobnosti da klijent kontroliše oblik odgovora.

Za IoT scenarije gde krajnji uređaji često traže samo nekoliko polja iz složenih telemetrijskih zapisa (npr. samo temperaturu iz strukture koja sadrži i vlažnost, pritisak, GPS koordinate i battery status), GraphQL i gRPC nude direktnu uštedu. REST se može optimizovati kroz definisane parcijalne reprezentacije (sparse fieldsets, npr. ?fields=temp,timestamp), ali to zahteva dodatnu logiku na serveru i nije deo standardne REST semantike.

6.3 Latencija i propusnost pod opterećenjem

Pri 10 VU sva tri protokola pokazuju sličnu prosečnu latenciju (1.3–2.7 ms), što sugerise da na niskom opterećenju režijski troškovi protokola (serijalizacija, header overhead) dominiraju nad razlikama u propusnosti. Razlike postaju jasne tek pri 100 i 500 VU, kada protokol mora da obrađuje paralelne zahteve i kada veličina payload-a počinje da utiče na zasićenost mreže i CPU-a klijenta.

U Scenariju A (write-intensive), REST je postigao najvišu propusnost (17,608 RPS pri 500 VU), nešto iznad gRPC-a (15,640 RPS) i znatno iznad GraphQL-a (7,273 RPS). GraphQL gubi performanse jer svaki zahtev prolazi kroz dodatni sloj parsiranja upita i resolver-a.

U Scenariju B (read s podskupom polja), gRPC preuzima (12,312 RPS pri 500 VU) zahvaljujući kombinaciji manjeg payload-a i HTTP/2 multipleksiranja, dok REST pada na 4,887 RPS upravo zbog over-fetching-a.

p95 vrednosti prate isti obrazac kao prosečne, ali su konzistentno 30–60% više. U svim stabilnim scenarijima (A i B) nijedan zahtev nije odbačen, što potvrđuje da je sistem ostao u stabilnom režimu rada.

6.4 Napomena o sistemskim resursima

Paralelno sa k6 testovima, prikupljani su podaci o iskorišćenosti CPU-a i RAM-a po Docker kontejneru (docker stats sa periodičnim uzorkovanjem). Mereni su servisni kontejner svakog protokola (iot_rest, iot_grpc, iot_graphql) i baza podataka (iot_postgres). Vrednosti CPU% su izražene u skali jednog jezgra (200% ≈ dva potpuno iskorišćena jezgra).

Iz dobijenih podataka izvlače se tri jasne opservacije:

- U stabilnim scenarijima (A i B), servisni kontejner ne predstavlja usko grlo — REST i gRPC servisi konzistentno koriste 180–220% CPU-a (oko dva jezgra), dok je GraphQL servis nešto lakši (100–130%), što je u skladu sa njegovom nižom propusnošću.
- Postgres je dominantna komponenta u potrošnji CPU-a: skalira se sa ~150% pri 10 VU na ~500% pri 100/500 VU u A/B, a u Scenariju C dostiže 950–1180% (10–12 jezgara). Upravo baza podataka je tačka zasićenja pri agregatnim operacijama, a ne sam protokol.
- U Scenariju C servisni kontejneri padaju na svega 3–25% CPU-a — server čeka na bazu i odbija zahteve u resolver-u/middleware-u pre nego što oni dospeju do upita. Ovo objašnjava zašto GraphQL u Scenariju C ima stopu grešaka od ~50% (ne 100%): polovina zahteva otpada na nivou parsiranja/resolver-a sa svega 3–6% CPU-a, dok preostala polovina pogađa zasićenu bazu.

Memorija nije bila ograničavajući faktor ni u jednom scenariju — servisni kontejneri zauzimaju 0.9–1.6% (oko 150–250 MB), a Postgres 2–10%. Tabela ispod sumira prosečne i maksimalne vrednosti po testu:

Test	App CPU avg	App CPU max	PG CPU avg	PG CPU max	App MEM	PG MEM
A — REST 10	180.4%	187.6%	156.6%	157.9%	1.45%	2.09%
A — REST 100	209.6%	217.6%	448.7%	471.7%	1.35%	3.32%
A — REST 500	212.7%	217.2%	477.6%	502.4%	1.51%	3.37%
A — gRPC 10	178.3%	184.4%	151.3%	159.6%	1.30%	3.52%
A — gRPC 100	192.6%	196.8%	415.3%	431.8%	1.44%	4.74%
A — gRPC 500	199.5%	224.4%	448.0%	502.8%	1.57%	4.78%
A — GraphQL 10	96.2%	98.4%	99.6%	101.3%	0.93%	4.93%
A — GraphQL 100	109.7%	112.2%	118.8%	122.3%	0.97%	4.36%
A — GraphQL 500	124.0%	127.8%	101.7%	106.6%	1.51%	3.59%
B — REST 10	106.4%	109.8%	409.4%	421.6%	1.16%	3.57%
B — REST 100	111.2%	120.2%	733.8%	782.0%	1.22%	3.95%
B — REST 500	119.3%	149.0%	872.9%	1079.5%	1.40%	3.25%
B — gRPC 10	192.7%	198.0%	198.1%	202.5%	1.24%	3.36%
B — gRPC 100	159.8%	163.2%	440.7%	465.0%	1.32%	4.40%
B — gRPC 500	174.6%	215.2%	534.7%	641.5%	1.61%	4.41%
B — GraphQL 10	103.4%	104.4%	105.4%	106.6%	0.91%	4.53%
B — GraphQL 100	107.5%	109.2%	98.6%	101.0%	0.96%	4.03%
B — GraphQL 500	127.1%	127.9%	86.8%	88.0%	1.53%	3.37%
C — REST 10	82.8%	85.6%	955.9%	977.1%	1.22%	4.24%
C — REST 100	142.5%	148.2%	1002.3%	1031.5%	1.39%	5.80%
C — REST 500	140.5%	149.6%	1023.5%	1081.1%	1.81%	5.26%
C — gRPC 10	2.7%	4.0%	1119.2%	1135.8%	1.53%	3.74%
C — gRPC 100	17.6%	36.9%	1021.7%	1175.9%	1.34%	9.95%
C — gRPC 500	23.9%	68.0%	1068.2%	1162.3%	1.63%	10.08%
C — GraphQL 10	6.2%	11.5%	1098.5%	1117.5%	0.86%	3.79%
C — GraphQL 100	4.3%	6.6%	1098.7%	1126.7%	0.95%	3.80%
C — GraphQL 500	3.5%	6.8%	1097.8%	1122.3%	1.00%	3.80%

7. Zaključak

Testovi su pokazali da ne postoji jedinstveno „najbolje“ rešenje — izbor između REST-a, GraphQL-a i gRPC-a zavisi od karakteristika konkretnog IoT scenarija. Ipak, iz dobijenih rezultata mogu se izvući jasne preporuke po tipu radnog opterećenja.

Za write-heavy operacije sa jednostavnim odgovorima (Scenario A — pojedinačno upisivanje očitavanja), REST ostaje pragmatičan izbor: najveća propusnost (17,608 RPS), najniža latencija pri svim opterećenjima, široka podrška u alatima i bibliotekama, i trivijalna integracija na strani uređaja. Gubitak u veličini odgovora (236 B vs 59 B za gRPC) je u apsolutnim brojevima mali, jer je odgovor sam po sebi kratak.

Za read-heavy operacije sa selektivnim poljima (Scenario B), gRPC je najbolji izbor: kombinacija Protobuf-a, HTTP/2 multipleksiranja i mogućnosti specifikacije polja daje 2.5× veću propusnost od REST-a i 7.8× manji volumen prenesenih podataka. GraphQL je dobar kompromis kada je gRPC infrastrukturno teško podržati (browser klijenti, restriktivni mrežni okviri) — zadržava prednost selektivnog dohvata uz konvencionalni HTTP/JSON transport.

Za računski intenzivne agregatne upite (Scenario C), sva tri protokola su pokazala ograničenja serverskog sloja, ne protokola samog po sebi. Visoke stope grešaka (>99% za REST/gRPC, ~50% za GraphQL) ukazuju na zasićenje servera (timeout-i, queue overflow, ograničenja resursa) pre nego na razlike u efikasnosti protokola. Rezultati iz Scenarija C treba interpretirati kao

informaciju o stabilnosti serverskog stack-a pod ekstremnim opterećenjem, a ne kao komparativnu prednost jednog protokola nad drugim. Za pouzdano upoređivanje performansi pod agregatnim opterećenjem, potrebno je prethodno otkloniti uska grla na serveru (povećati timeout-e, optimizovati agregacione upite u bazi i koristiti hourly_aggregates view, dodati horizontalno skaliranje).

U širem IoT kontekstu, gde se uloga klijenta sve češće seli na rubne uređaje sa ograničenim resursima, prednosti binarnih protokola (Protobuf/gRPC) — manji payload, niža potrošnja energije po zahtevu, deterministička šema — postaju značajnije od fleksibilnosti tekstualnih formata. Međutim, kompleksnost gRPC alata, potreba za .proto definicijama na svim klijentima, i slabija podrška u browser-baziranim okruženjima i dalje čine REST i GraphQL pragmatičnim izborom za hibridne arhitekture u kojima se uređaji povezuju i preko web frontenda. Optimalan dizajn često podrazumeva kombinaciju: gRPC između rubnog uređaja i edge gateway-a, REST ili GraphQL između gateway-a i cloud sloja.